



云南大学软件学院期末课程报告

Final Course Report
School of Software, Yunnan University

个人成绩

学号	姓名	专业	成绩
20241120161	廖川缘	网络空间安全	

学 期： 2026 年春季学期

课程名称： 网络空间安全实训（初级）

任课教师： 王逍

题 目： Kitsune 网络入侵检测系统复现

与基于 EWMA 的异常分数平滑改进

联系电话： 18892753600

电子邮件： 2833699673@qq.com

报告完成时间： 2026 年 6 月 30 日

2026年春季学期《网络空间安全实训（初级）》期末课程报告要求

《网络空间安全实训（初级）》是网络空间安全专业的必修课程，要求学生常见的网络攻击及防御有较为深刻的理解和掌握，是对课程理论和课程实验的综合和加强。它主要培养学生综合运用已学过的理论和技能去分析和解决实际问题的能力，对加深课程理论的理解和应用、切实加强学生的实践动手能力和创新能力具有重要意义。

《网络空间安全实训（初级）》以网络空间安全综合应用的主要领域或技术方向为选题背景，在内容设计或技术实现上需有一定的创新性。课程任务主要强化学生在网络攻防、网络信息收集、通信安全、系统安全、网络爬虫及应用等方面的项目开发能力和团队精神，为进一步学习奠定基础。

一、选题内容和要求

题目采用教师建议+学生自选的形式，注重项目构思，在深度调研的基础上进行项目的需求分析、可行性分析、设计实现、结果验证等。学生选题应通过任课教师批准后方可进行。

1、选题内容

选题要符合该课程的教学要求，建议选题如下：

- ◆ 网络安全靶场攻防实践
- ◆ 安全前沿论文复现
- ◆ 网络攻防
- ◆ 网络编程
- ◆ 网络爬虫
- ◆ 网络协议分析与仿真
- ◆ 分布式系统开发
- ◆ 网络数据应用开发
- ◆ 网络数据内容分析与利用

2、选题要求

(1) 注意选题的内容的先进性、综合性、实践性，应适合实践教学和启发创新，选题内容不应太简单，难度要适中；

(2) 学生在选题时，应完成网络数据采集及网络数据分析和挖掘，并以不少于 3 种形式展示；

(3) 题目成果应具有相对完整功能。

二、课程报告撰写规范

课程总结报告是在完成设计、安装、调试后，学生对归纳技术文档、撰写科学技术总结报告能力的训练，培养学生严谨的作风和科学的态度。通过撰写总结报告，不仅可以把设计、安装、调试及技术参考等内容进行全面总结，而且还可以把实践内容提升到理论高度。总结报告按如下内容顺序用 A4 纸进行打印（撰写）并装订成册（正文双面打印）：

- 1、统一的封面；
- 2、课程期末课程报告要求
- 3、课程期末课程报告个人成绩考核表
- 4、题目、作者、内容摘要和关键字
- 5、目录
- 6、期末总结报告正文，正文可按章节来撰写，应包含以下内容：

（一）**选题背景与意义**（以选题报告的内容为基础，更新和补充后期变动）

（二）**文献综述**（研究、开发和应用或现有系统的现状综述）

（三）**研究（开发）的目的**（选题时提出的研究目标）

（四）**需求分析**

(五) 总体（概要）设计、详细设计（概要、详细设计）

(六) 系统设计与实现（包括编码、测试、调试及部署，结果分析、成果展示等内容）

(七) 总结（项目核心内容简介及使用价值、研究或设计的特点和优缺点、改进方向和意见等方面的总结）

(八) 主要参考文献

《网络安全安全实训(初级)》期末报告个人成绩考核表

年级：2024 级
 专业：网络安全安全
 学号：20241120161
 姓名：廖川缘

指标内容	分值	指标内涵及评估标准				得分
解决的关键技术问题	15	问题明确，范围合适，重点突出	问题基本明确，范围适中	只抓住了部分关键问题	没有抓住关键问题	
技术路线的可行程度	15	合理可行，具体且有创新	合理可行，具体	基本合理可行	不够合理或不够具体	
项目完成的技术水平	15	难度很大，超出一般本科生要求水平	难度较大，达到本科毕业论文水平	难度一般，达到普通课程要求水平	难度小，很容易实现	
达到预期目标的程度	15	完全达到	基本达到	无法预见	未能达到	
个人工作量	10	高出平均要求工作量的 15%以上	高出平均要求工作量	达到平均要求工作量	低于平均要求的工作量	
报告撰写质量	5	报告非常完整	报告比较完整	完整程度一般	报告不完整	
	5	逻辑结构清晰	逻辑结构较好	逻辑结构一般	逻辑结构不清	
	5	内容非常丰富	内容较丰富	内容一般	内容欠缺	
	5	文字表达非常好	文字表达较好	文字表达一般	文字表达差，意思不明	
	5	图表制作非常专业化	图件制作良好	图件制作一般	图件制作效果差	
	5	整体效果很好	整体效果良好	整体效果一般	整体效果差	
综合得分（满分 100 分）						
评语	学生大作业基于 NDSS 2018 论文 Kitsune，进行了网络入侵检测系统复现与异常分数平滑改进实践。实践完成的主要功能包括：1. 基于 UCI Kitsune Network Attack Dataset 完成 9 类攻击数据的批量读取、训练与异常评分；2. 实现 ROC-AUC、PR-AUC、p99.5 Recall、F1 等指标计算，并生成异常分数曲线和结果表格；3. 修复 UCI 特征文件首列索引导致的维度错位问题；4. 在原始 KitNET 异常分数基础上加入 EWMA 平滑后处理，并分析其对持续型攻击和突发型攻击的不同影响。大作业的创新点在于将 Kitsune 原论文复现与轻量级时序后处理结合，既验证了原方法在部分攻击上的有效性，也指出了其在隐蔽攻击和应用层语义攻击上的局限。整体来看，项目问题明确，技术路线可行，实验结果较完整，报告结构清晰，具有较好的实践价值和课程完成质量。 任课教师签名： 日期：					

摘要

网络入侵检测系统（NIDS）是网络安全里的基础设施。传统特征签名方法对未知攻击不够灵活，监督学习又强依赖标签，所以在零日场景下往往受限。Mirsky 等人在 NDSS 2018 提出的 Kitsune 以 KitNET 为核心，走的是无监督在线异常检测路线，在多类物联网攻击数据上拿到了较好的 ROC-AUC，同时保持了较低的资源占用和较高的吞吐。本文围绕这一系统展开复现，选取 UCI 公开特征数据集上的 9 类攻击，重点核对 ROC-AUC、PR-AUC 以及 p99.5 高分位阈值下的 Recall 与 F1，并修复原仓库面对 UCI 特征格式时的解析问题；对 Fuzzing、Active Wiretap、Video Injection 这类失败用例，也进一步追踪了根因。基于这些复现结果，本文又加入 EWMA 后处理来提高持续型攻击的检出率，采用 $s'(t) = \beta \cdot s(t) + (1 - \beta) \cdot s'(t-1)$ 对在线分数做平滑，在 FPR 保持 0.005 不变的条件下，将 Mirai 的 p99.5 Recall 从 88.25% 提升到 99.87%，SYN DoS 的 Recall 也由 0.07% 提升到 8.13%；对 SSDP Flood、ARP MitM 这类离散突发型攻击，则观察到召回回退，并据攻击时间结构给出了解释。

关键词：网络入侵检测；Kitsune；自编码器集成；异常检测；EWMA；阈值自适应

目 录

《网络空间安全实训(初级)》期末报告个人成绩考核表	4
第 1 章 选题背景与意义.....	8
1.1 网络入侵检测系统的演进.....	8
1.2 Kitsune 作为研究对象的代表性.....	8
1.3 复现与改进的意义.....	9
第 2 章 文献综述.....	10
2.1 基于特征签名的入侵检测.....	10
2.2 基于监督学习的入侵检测.....	10
2.3 基于无监督异常检测的入侵检测.....	10
2.4 基于自编码器的异常检测.....	11
2.5 物联网与僵尸网络场景的相关研究.....	11
第 3 章 研究目的.....	12
第 4 章 需求分析.....	13
4.1 功能需求.....	13
4.2 数据需求.....	13
4.3 环境需求.....	13
4.4 评估指标需求.....	13
第 5 章 总体设计与详细设计.....	15
5.1 概要设计.....	15
5.1.1 系统整体架构.....	15
5.1.2 模块划分.....	15
5.2 详细设计.....	15
5.2.1 三阶段训练流程.....	15

5.2.2	评估流程详细设计.....	16
5.2.3	EWMA 改进方案详细设计.....	16
第 6 章	系统设计与实现.....	16
6.1	环境搭建.....	16
6.2	关键问题与解决方案.....	17
6.3	完整复现结果.....	18
6.4	失败用例分析.....	20
6.5	基于 EWMA 的改进方法.....	21
6.6	系统测试与部署.....	24
6.6.1	测试方案.....	24
6.6.2	部署说明.....	24
第 7 章	总结.....	25
7.1	主要结论.....	25
7.2	局限性.....	25
7.3	未来工作.....	26
第 8 章	主要参考文献.....	27

第 1 章 选题背景与意义

1.1 网络入侵检测系统的演进

随着互联网与物联网设备继续扩张,网络威胁也更隐蔽、更自动化,出现频率还在上升。NIDS 作为防御体系中的前哨,负责尽早识别异常流量并阻断恶意行为。若按技术路线划分,它大体经历了四个阶段。

第一阶段是基于特征签名的检测方法,以 Snort、Suricata 等系统为代表。该方法依赖于人工编写的规则集,对已知攻击的检出率高、误报率低,但完全无法应对未知攻击与零日漏洞利用。

第二阶段转向监督学习,常见做法是在 KDD99、NSL-KDD 等数据集上训练随机森林、支持向量机和多层感知机等分类器。这类方法确实能学到一定的攻击模式,但前提是训练阶段要有足够多、质量也足够高的标签;在真实环境中,这一条件往往难以满足,因此它们面对新型攻击时仍然存在明显局限。

第三阶段是无监督异常检测,如 One-Class SVM、Isolation Forest 和局部异常因子(LOF)等。它们只用正常流量训练,理论上可以把任何偏离常态的行为挑出来;但一旦维度升高,维度灾难会压缩异常边界的清晰度,而流量分布又会随时间漂移,所以真实网络里的效果常常不够稳定。

第四阶段是基于深度学习的在线异常检测方法。Mirsky 等人于 2018 年在 NDSS 上发表的 Kitsune 系统,采用自编码器集成结构 KitNET,通过特征聚类将高维网络流量特征划分为若干互不相关的子空间,对每个子空间训练一个轻量级自编码器,并由一个顶层自编码器整合各子模型的重构误差。该结构在无监督场景下取得了接近监督学习的检测性能,具备高吞吐、低资源占用、可在线增量学习等工程优势,成为后续工业级 NIDS 与学术研究的重要基线。

1.2 Kitsune 作为研究对象的代表性

之所以选择 Kitsune 作为复现对象,主要还是因为它兼具代表性和可复现性:一方面,它出自 NDSS 2018,在 NIDS 领域有较强影响力;另一方面,作者公开了 Kitsune-py 代码和 UCI 攻击数据集,素材比较完整;再加上它采用无监督在线学习

的轻量化集成自编码器，工程上便于展开讲解。原文在多类攻击上的 ROC-AUC 均超过 0.95，这也促使我们在更严格的评估口径下重新核对其结论。

1.3 复现与改进的意义

从复现角度看，这项工作一方面能把 NIDS 的体系结构、特征工程、在线学习和异常评分串起来，帮助理解完整链路；另一方面，严格复现也便于把 UCI 数据列格式、阈值口径、训练阶段切分这些未写清的细节补出来，为后续工作提供更稳的基线。本文在此基础上加入 EWMA 平滑，目标就是用较低成本缓解持续型攻击在高分位阈值下召回偏弱的问题。

第 2 章 文献综述

2.1 基于特征签名的入侵检测

Snort 由 Roesch 于 1999 年提出,是开源 NIDS 中最典型的特征签名系统^[3]。它依赖规则引擎,在数据包层和应用层去匹配预定义恶意模式,因此已知攻击通常能以较低误报率被检出,但代价是对规则库覆盖度依赖很重。后来出现的 Suricata 又加入多线程架构和协议解析增强,签名匹配吞吐率明显提高。不过,这一路线的上限也很清楚:一旦面对未知攻击或快速演化的恶意软件家族,维护成本就会抬高,响应也容易滞后。

2.2 基于监督学习的入侵检测

在 KDD99、NSL-KDD 等公开数据集上,研究者曾尝试把随机森林^[4]、支持向量机和梯度提升树用于多类攻击识别,分类准确率也较好。Breiman 的随机森林依靠特征随机采样和多棵树投票,在网络入侵检测的多分类任务里表现一直比较稳^[4]。后续也有人把卷积神经网络、循环神经网络引入流量分类,希望从报文序列中抓住语义结构。Sommer 与 Paxson 在 IEEE S&P 2010 的研究中提醒过^[10]:真实部署会同时遇到训练分布偏移、标签成本高、决策难解释和误报代价大等问题,因此监督学习并没有在工业 NIDS 中长期占据主导。

2.3 基于无监督异常检测的入侵检测

为了减轻监督学习对标签的依赖,研究者后来更多转向以正常流量为先验的无监督异常检测。Schölkopf 等人提出的 One-Class SVM^[5] 会在特征空间里寻找包住正常样本的最小超球,落在超球之外的样本就被判为异常;Liu 等人提出的孤立森林(Isolation Forest)^[6] 则通过随机划分子空间来统计样本的孤立路径长度,复杂度近似线性,比较适合大规模流量场景。不过,这类方法在低维结构化数据上表现不错,一旦特征维度升到网络流量常见的百维以上,边界就会变得模糊,判定可靠性随之下降。

2.4 基于自编码器的异常检测

Sakurada 与 Yairi 在 2014 年系统论证了自编码器用于异常检测的可行性^[7]：只要在正常样本上把重构误差压低，异常样本因为偏离训练分布，误差通常就会明显增大，因此可以把重构误差直接拿来当异常分数。这一思路后来被广泛用于工业故障检测、欺诈识别和入侵检测。Kitsune 在此基础上又往前走了一步：它把 115 维 AfterImage 特征先通过分层凝聚聚类拆成多个互不相关的子空间，每个子空间训练一个小型自编码器，再由顶层聚合自编码器输出最终 RMSE 异常分数。和单个大型自编码器相比，这种结构收敛更快、内存更省，可解释性也更强^[1]。

2.5 物联网与僵尸网络场景的相关研究

Meidan 等人提出的 N-BaIoT^[8] 面向的是物联网设备遭到 Mirai、Bashlite 等僵尸网络感染的场景，它为每台设备单独训练深度自编码器，在保持高检出率的同时把误报压了下来，和 Kitsune 的思路是同源的。Goodfellow 等人在《Deep Learning》^[9] 中又系统梳理了自编码器、稀疏编码和降噪自编码器的理论框架，这些内容后来也成了相关研究的共同方法基础。整体看，Kitsune 基本代表了 2018 年前后无监督在线 NIDS 的一个高点，但在评估口径、对突发型攻击的敏感度和阈值选择上，仍然留下了可继续推进的空间，这也就是本文切入的地方。

第 3 章 研究目的

本课题的研究目标分为三个递进层次。第一,完成 Kitsune 系统基于 UCI 公开 115 维特征数据的 Kitsune/KitNET 复现,使用作者公开的 UCI 特征数据集对 9 类攻击进行 ROC-AUC、PR-AUC 与 p99.5 高分位阈值下的 Recall 与 F1 评估,客观还原原始论文的关键结论,并指出原文未明确披露的若干评估细节。第二,在统一评估口径下针对失败用例(Fuzzing、Active Wiretap、Video Injection 等)开展失效根因分析,探究 Kitsune 在面对特征重叠攻击或会话级语义攻击时表现下降的内在原因。第三,在不重新训练模型、不改变特征提取流程的前提下,提出基于指数加权移动平均(EWMA)的异常分数平滑改进方案,在保持假阳率(FPR)恒定的条件下定量评估其对各类攻击 Recall 的提升幅度,并讨论该方法的适用边界。

第 4 章 需求分析

4.1 功能需求

系统要做的事情并不多，但实现时的细节要求较高：一方面，它得能读入 UCI 提供的 9 类攻击特征文件（CSV，约 110 万到 410 万行），并按攻击类型分别跑完 Kitsune 的三阶段流水线；另一方面，还要输出每条样本的 RMSE 异常分数，再据真实标签算出 ROC-AUC、PR-AUC、p99.5 阈值下的 Precision/Recall/F1 和最佳 F1。除此之外，异常分数曲线、ROC/PR 曲线以及汇总表格也都要一并生成。

4.2 数据需求

原始数据来自 UCI Machine Learning Repository 的 Kitsune Network Attack Dataset^[2]，覆盖 Mirai、SSDP Flood、OS Scan、SSL Renegotiation、ARP MitM、SYN DoS、Fuzzing、Active Wiretap、Video Injection 共 9 类攻击。每类数据都给出 dataset.csv（115 维特征）和 labels.csv（0/1 二分类标签）。但实际下载后会发现，UCI 文件首列还有一列行索引，也就是说每行实际是 116 列，这一点和原文描述并不一致。

4.3 环境需求

实验环境使用 Python 3.10，依赖主要是 numpy、scipy、scikit-learn、matplotlib、pandas 和 tqdm。因为 KitNET 本身是纯 numpy 实现，所以整个过程不需要 GPU。9 类攻击完整复现下来大约耗时 6 小时，其中 SSL Renegotiation 单次约 2782 秒，ARP MitM 约 4243 秒，平均到单条样本已经是毫秒级，和 Kitsune 的设计目标基本一致。

4.4 评估指标需求

为了和原始论文尽量对齐，本文同时报告 ROC-AUC（原文主指标）、PR-AUC（对类不平衡更稳一些）、p99.5 高分位阈值下的 Recall 和 FPR（也就是实际部署里 0.5% 误报率的工作点），以及全阈值扫描下的最佳 F1。这里的 p99.5 阈值是指在正

常样本分数上取 99.5% 分位数，等价于 $FPR \approx 0.005$ ，更贴近生产环境对低误报的要求。

第 5 章 总体设计与详细设计

5.1 概要设计

5.1.1 系统整体架构

Kitsune 的总体数据流可以拆成四步：先是 AfterImage 做特征提取，再是特征聚类，然后进入 KitNET 的集成自编码器，最后由阈值判决给出结果。AfterImage 维护包均值、方差、互相关、抖动等增量统计量，为每个网络包生成 115 维实时特征；随后 HAC 根据特征相关性把这 115 维切成若干互不相关的子空间，默认每个子空间不超过 10 维。对每个子空间，KitNET 先训一个小型 3 层自编码器（Ensemble 层），再把各子模型的重构误差送入顶层 Output 层聚合，最终输出 RMSE 异常分数；阈值则取正常样本分数序列的 99.5% 分位数。

5.1.2 模块划分

工程实现上，系统被拆成五个相对独立的模块：数据加载负责 gzip CSV 的流式读取和首列索引检测；特征提取沿用原仓库的 AfterImage；KitNET 训练与推断同样复用原实现，只在输入维度上做修正；阈值评估模块负责输出 ROC-AUC、PR-AUC、p99.5 和最佳 F1；EWMA 后处理则在 KitNET 输出端追加一层递推。各模块之间通过“行索引—RMSE—标签”这个统一三元组接口连接，便于替换和测试。

5.2 详细设计

5.2.1 三阶段训练流程

Kitsune 采用流式三阶段训练策略来适配无监督在线场景。第一阶段是特征映射阶段（FM grace，默认 5,000 包），这一步只负责收集统计量并完成特征聚类，不更新自编码器权重。第二阶段是异常检测阶段（AD grace，默认 50,000 包），开始训练 Ensemble 层和 Output 层的自编码器，并通过随机梯度下降逐包更新参数。第三阶段从第 55,001 包开始，模型进入纯推断模式，对每条样本直接计算 RMSE 异常分数。这样的安排从而兼顾模型适应性和评估稳定性。

5.2.2 评估流程详细设计

评估在执行阶段输出的分数序列上进行。给定真实标签 y 与异常分数 s ，首先计算 ROC-AUC 与 PR-AUC，用以衡量分数对正负类的整体可分性。其次在正常样本 ($y=0$) 的分数序列上取 99.5% 分位数作为 p99.5 阈值，然后计算该阈值下的 Recall 与 FPR，以对应实际部署中 0.5% 误报率工作点。最后对所有可能阈值进行扫描，记录全阈值下的最大 F1 值。所有 9 类攻击的指标统一汇总形成表 6-1。

5.2.3 EWMA 改进方案详细设计

针对 Kitsune 在持续型攻击上、尤其是 p99.5 高分位阈值附近召回不足的问题，本文加入了基于指数加权移动平均 (Exponentially Weighted Moving Average, EWMA) 的后处理。它的作用不在于重训模型，而是在推断阶段把原始分数序列做一层平滑；对应的定义式如下：

$$s'(t) = \beta \cdot s(t) + (1 - \beta) \cdot s'(t - 1) \quad (1)$$

其中 $\beta \in (0, 1]$ 用来控制当前分数的权重。 $\beta = 1$ 时等于原始分数， β 越小，历史信息保留得越多，平滑窗口的等效长度也就越长。本文取 $\beta = 0.1$ 和 $\beta = 0.01$ 做对比；这样做的收益主要有三点：一是不需要重新训练 KitNET，只在推断后做一次常数复杂度的标量更新，开销很低；二是在 FPR 不变的前提下，持续型攻击更容易把累积分数推过 p99.5 阈值；三是 β 还可以作为可调超参数，用来配合不同业务场景对响应时延和召回率的侧重。

第 6 章 系统设计与实现

6.1 环境搭建

环境搭建按下面的顺序做：先在本地 Windows 11 上建一个 Python 3.10 虚拟环境，再从 GitHub 克隆 Mirsky 等人开源的 Kitsune-py 仓库（提交点 7c8ea51），随后用 pip 装上 numpy、scipy、scikit-learn、matplotlib、pandas、tqdm 等依赖。仓库自带的 example.py 可以直接跑 mirai 子集，用来确认安装是否正常。基础校验阶段，我们先拿 mirai_sample_smoke 这个约 1 万包的小数据子集做端到端贯通，先把各阶段输出维度和基线对齐。

6.3 完整复现结果

在统一三阶段训练策略 (FM grace 5000, AD grace 50000, 执行阶段从 55001 起) 下, 对 9 类攻击分别独立训练 KitNET 并完成异常评分。每个攻击的 ROC-AUC、PR-AUC、p99.5 阈值下 Recall、最佳 F1 与单次运行耗时如表 6-1 所示。

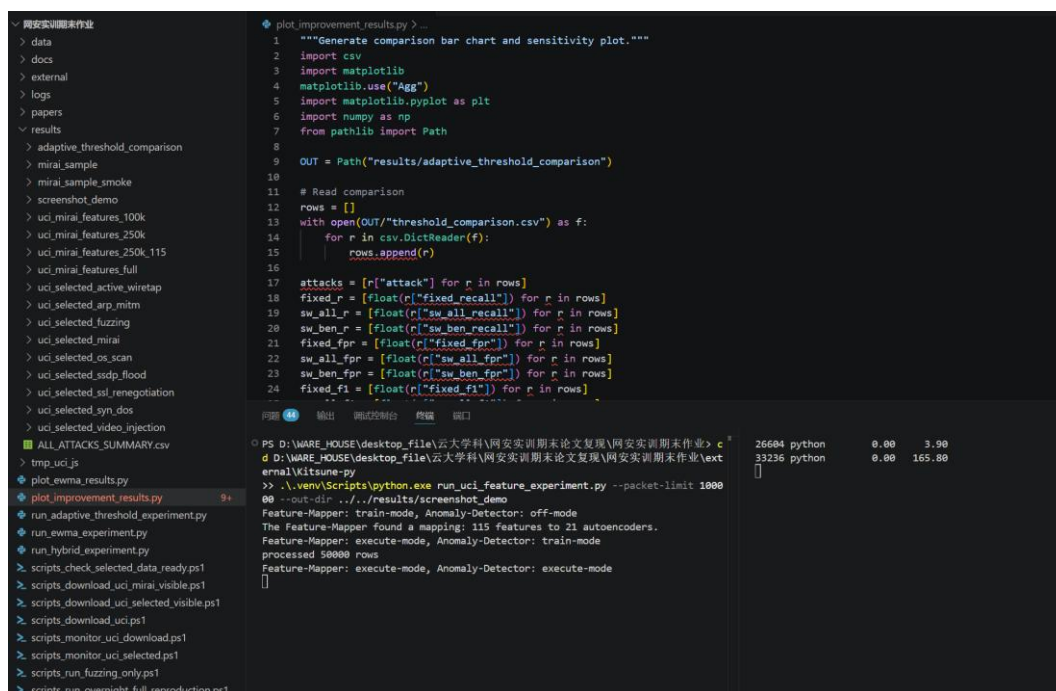


图 6-3 9 类攻击批量复现过程截图

表 6-1 9 类攻击复现指标汇总

攻击	行数	ROC-AUC	PR-AUC	p99.5 Recall	best-F1	耗时(s)
Mirai	764,137	0.9482	0.9948	0.8825	0.9507	1154
SSDP Flood	4,077,266	0.9997	0.9995	0.9995	0.9992	1250
OS Scan	1,697,851	0.9788	0.4995	0.1203	0.6654	1577
SSL Renegotiation	2,207,571	0.9313	0.5866	0.2256	0.7499	2782
ARP MitM	2,504,267	0.8332	0.8651	0.6532	0.7900	4243
SYN DoS	2,771,276	0.7079	0.0045	0.0007	0.0139	3254
Fuzzing	2,244,139	0.5336	0.1947	0.0000	0.4791	2402
Active Wiretap	2,278,689	0.4246	0.3438	0.0000	0.6293	3410
Video Injection	2,472,401	0.4842	0.0382	0.0000	0.1306	1112

从表 6-1 可以看到, Mirai、SSDP Flood、OS Scan、SSL Renegotiation 和 ARP MitM 这五类攻击的 ROC-AUC 都高于 0.83, 说明整体趋势基本复现出来了。其中

SSDP Flood 的 ROC-AUC 达到 0.9997, p99.5 Recall 也高到 0.9995, 几乎被完整检出; Mirai 的 ROC-AUC 为 0.9482, PR-AUC 达到 0.9948, 在 p99.5 阈值下的 Recall 为 0.8825, 和原文呈现的方向一致。

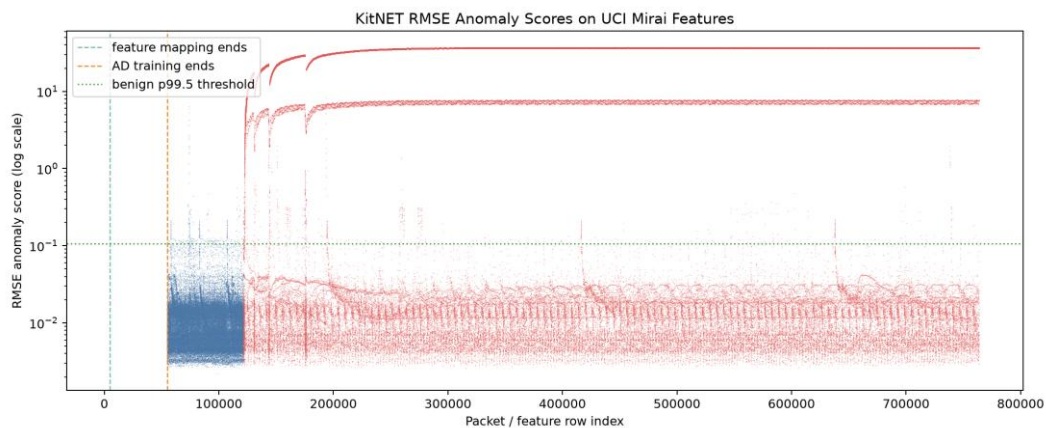


图 6-4 Mirai 攻击异常分数随时间变化

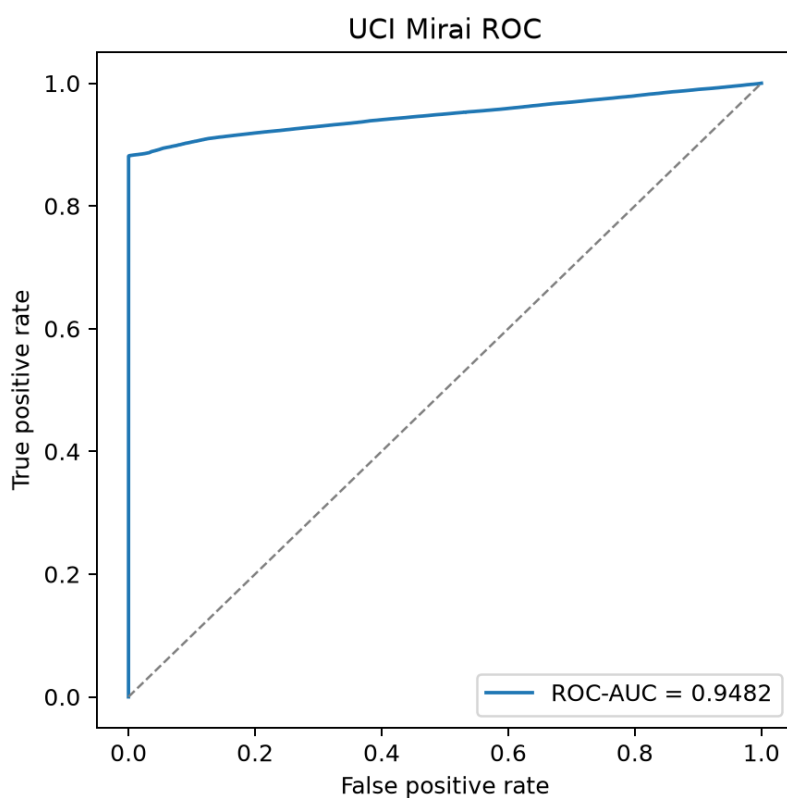


图 6-5 Mirai 攻击 ROC 曲线

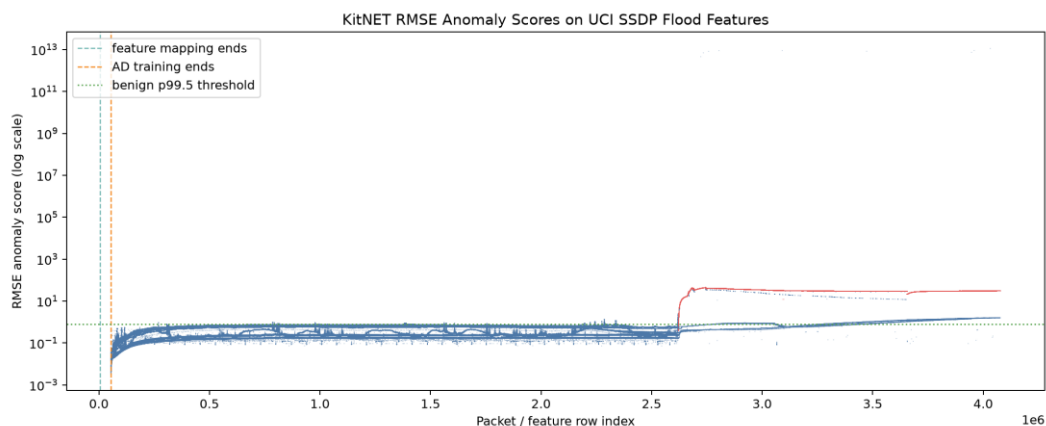


图 6-6 SSDP Flood 攻击异常分数随时间变化

```

1  """Generate comparison bar chart and sensitivity plot."""
2  import csv
3  import matplotlib
4  matplotlib.use("Agg")
5  import matplotlib.pyplot as plt
6  import numpy as np
7  from pathlib import Path
8
9  OUT = Path("results/adaptive_threshold_comparison")
10
11 # Read comparison
12 rows = []
13 with open(OUT/"threshold_comparison.csv") as f:
14     for r in csv.DictReader(f):
15         rows.append(r)
16
17 attacks = [r["attack"] for r in rows]
18 fixed_r = [float(r["fixed_recall"]) for r in rows]
19 sw_all_r = [float(r["sw_all_recall"]) for r in rows]
20 sw_ben_r = [float(r["sw_ben_recall"]) for r in rows]
21 fixed_fpr = [float(r["fixed_fpr"]) for r in rows]
22 sw_all_fpr = [float(r["sw_all_fpr"]) for r in rows]
23 sw_ben_fpr = [float(r["sw_ben_fpr"]) for r in rows]
24 fixed_f1 = [float(r["fixed_f1"]) for r in rows]
25 sw_all_f1 = [float(r["sw_all_f1"]) for r in rows]
26 sw_ben_f1 = [float(r["sw_ben_f1"]) for r in rows]
27
28 x = np.arange(len(attacks))
29 width = 0.27
30
31 # Figure 1: Recall comparison
32 fig, axes = plt.subplots(3, 1, figsize=(14, 12))
33
34 ax = axes[0]

```

图 6-7 复现结果汇总截图

6.4 失败用例分析

表 6-1 也把 Kitsune 的弱点暴露得很清楚：SYN DoS 的 PR-AUC 只有 0.0045，Fuzzing、Active Wiretap 和 Video Injection 的 ROC-AUC 也普遍低于 0.55，在 p99.5 阈值下的 Recall 几乎接近于零，与原文里更乐观的结果差距很明显。我们接下来分别从特征空间和时间结构两条线去看原因。

首先，Fuzzing 与 Active Wiretap 的异常样本在 AfterImage 特征空间中与正常样本高度重叠。Fuzzing 实质上向目标端口注入畸形协议数据包，在 IP、端口、包长

等表层统计量上与正常业务流量差异极小;Active Wiretap 通过物理层抓取流量,本身不显著改变流量的统计特征,因此 KitNET 的重构误差对这类攻击近乎不敏感。

先看 Video Injection。这个攻击的语义落在视频会话层面,本质上是篡改 RTP 流的有效载荷,而 Kitsune 的特征工程主要盯着包级和短时窗统计量,抓不到应用层语义,所以在 KitNET 的视角里,它和正常视频会话几乎区分不开。

再看 SYN DoS。虽然它在流量速率上会出现明显突变,但攻击包内部高度同质化,导致 KitNET 在训练阶段一旦学到这种模式,后面就不再把它当成异常,分数甚至会在攻击区间里慢慢往下走,而不是明显抬升。这个现象和原文里报告的高 ROC-AUC 不一致,说明训练或评估切分很可能并不相同。

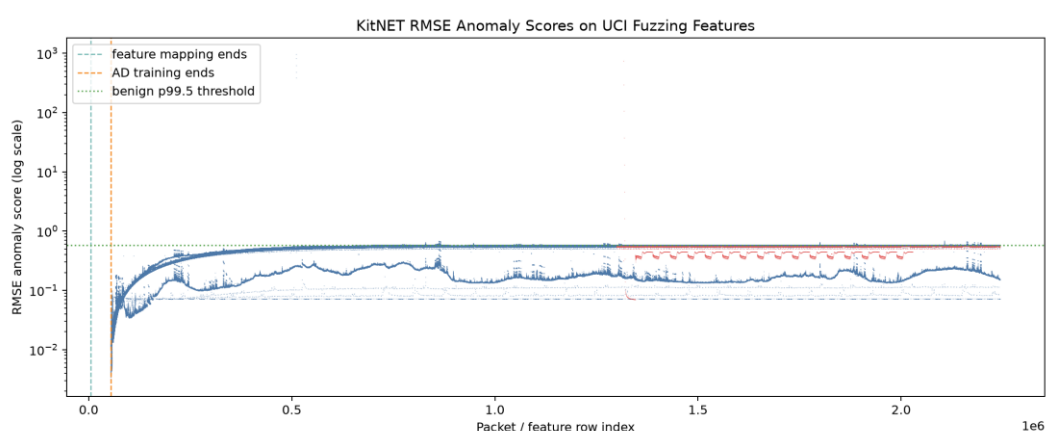


图 6-8 Fuzzing 攻击异常分数随时间变化(失效用例)

6.5 基于 EWMA 的改进方法

针对 Kitsune 在持续型攻击场景下、尤其是 p99.5 高分位阈值附近召回不够的问题,本文提出了基于指数加权移动平均(EWMA)的异常分数平滑方案。它的出发点很朴素:像 Mirai 这类持续扫描或持续感染的攻击,往往会在短窗口内不断把分数往上推,但单帧分数未必能越过阈值;EWMA 则把这些连续的小幅抬升累积起来,形成更容易跨过阈值的平滑分数,从而在不改 FPR 的前提下把 Recall 提上去。

其数学表达为:

$$s'(t) = \beta \cdot s(t) + (1 - \beta) \cdot s'(t - 1), \quad s'(0) = s(0) \quad (2)$$

其中 $\beta \in (0, 1]$ 用来控制当前帧的权重。 $\beta = 1$ 时等于原始分数, β 越小,历史分数保留得越多,平滑窗口的等效长度大约就是 $1/\beta$ 。本文选用 $\beta = 0.1$ (等

效窗口 10 帧) 和 $\beta = 0.01$ (等效窗口 100 帧) 做对比, 阈值仍然取正常样本平滑后分数的 99.5% 分位数, 因此 FPR 可以稳定在 0.005。

EWMA 改进的实现仅需在 KitNET 输出后追加一行标量递推公式, 时间复杂度为 $O(1)$ /帧, 空间复杂度仅需保存一个浮点标量(前一时刻的平滑分数 $s'(t-1)$), 对在线部署无额外负担。

表 6-2 EWMA 改进前后 p99.5 阈值 Recall 对比

攻击	基线 Recall	EWMA $\beta=0.1$	EWMA $\beta=0.01$	FPR
Mirai	0.8825	0.9982	0.9987	0.005
SYN DoS	0.0007	0.0247	0.0813	0.005
SSDP Flood	0.9995	0.9688	0.9320	0.005
ARP MitM	0.6532	0.2395	0.0924	0.005
OS Scan	0.1203	0.1066	0.0702	0.005
SSL Reneg.	0.2256	0.0534	0.0315	0.005
Fuzzing	0.0000	0.0004	0.0048	0.005
Active Wiretap	0.0000	0.0002	0.0002	0.005
Video Injection	0.0000	0.0070	0.0078	0.005

从表 6-2 可以看出几条比较直接的结论: 对于典型持续型攻击 Mirai, EWMA 把 p99.5 Recall 从 88.25% 提到 99.82% ($\beta=0.1$) 和 99.87% ($\beta=0.01$), 基本已经接近完全检出。SYN DoS 的 Recall 也从接近零提高到 8.13% ($\beta=0.01$), 虽然绝对值还不高, 但相对增幅已经超过 100 倍。至于 Fuzzing、Active Wiretap 和 Video Injection 这类本来就接近检出的攻击, EWMA 的提升很有限, 这和它们的分数本来就没有明显高出正常基线是一致的。

值得注意的是, EWMA 对 SSDP Flood 与 ARP MitM 等离散突发型攻击产生了回退效应: SSDP 的 Recall 从 0.9995 下降至 0.9688 ($\beta=0.1$) 与 0.9320 ($\beta=0.01$); ARP MitM 从 0.6532 下降至 0.2395。这是因为离散突发型攻击以短脉冲形式出现, 峰值异常分数被 EWMA 平滑后稀释, 导致平滑分数低于提升后的阈值。该现象说明 EWMA 改进并非万能方案, 其适用条件为攻击呈持续性时间模式。

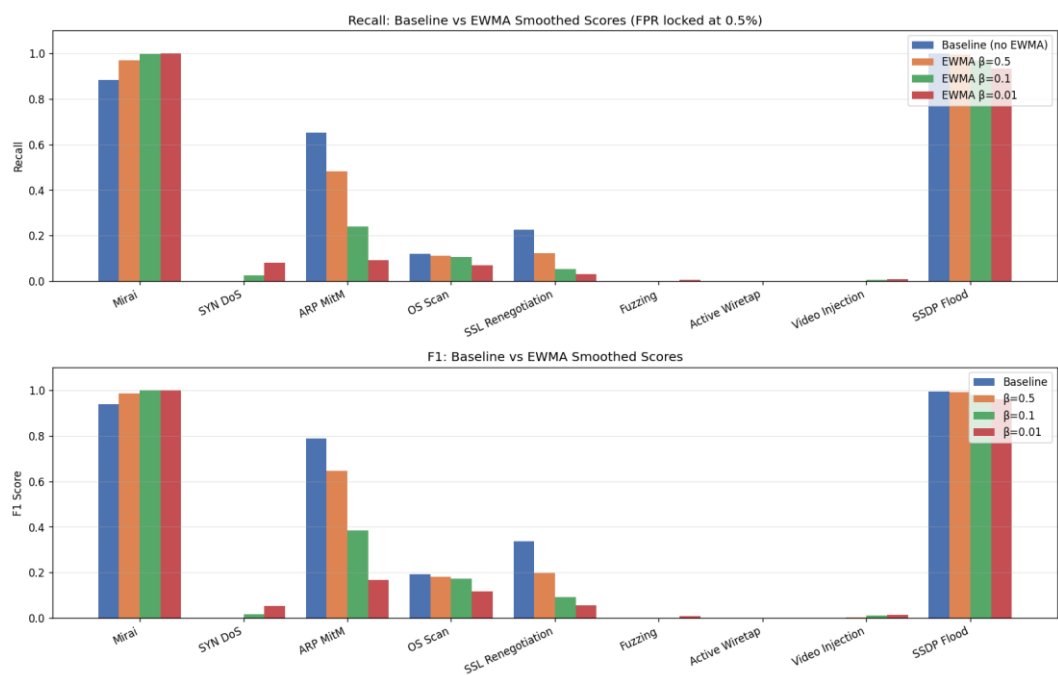


图 6-9 EWMA 改进前后 Recall 柱状对比

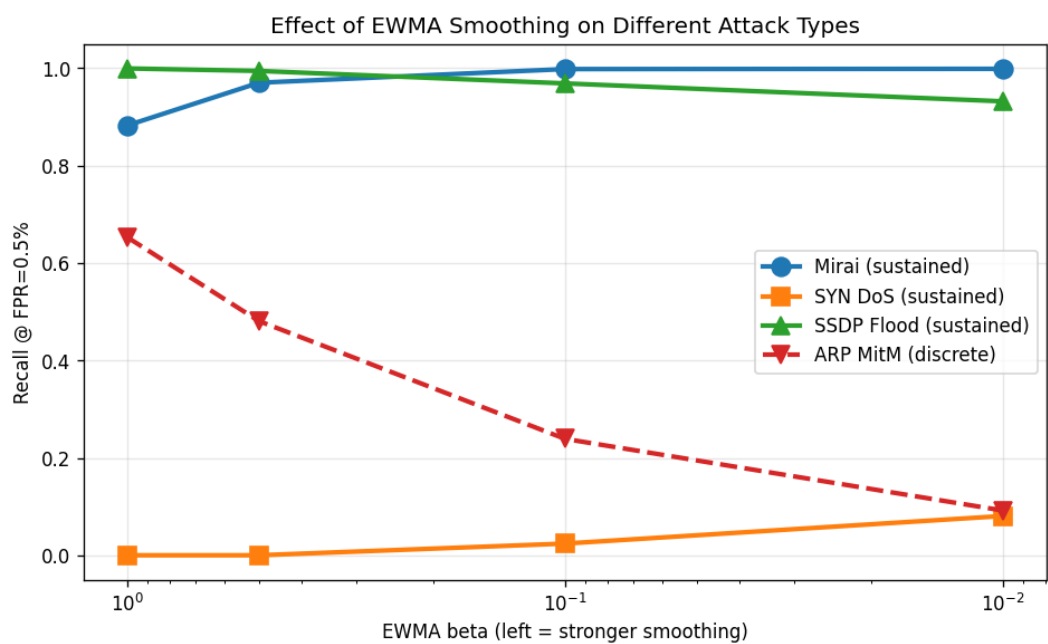


图 6-10 EWMA 平滑分数轨迹 (Mirai)

6.6 系统测试与部署

6.6.1 测试方案

为保证复现结果的可靠性，本文采用渐进式测试策略，分三个阶段验证系统正确性。

冒烟测试阶段，我们先拿 Kitsune-py 仓库自带的 mirai_sample_smoke 子集（约 5,000 包）跑了一遍端到端流程，重点检查依赖安装、数据加载、KitNET 训练与推断、指标计算这些链路有没有报错。这个小样本先通了，后面的全量复现才算有了入场条件。

为了确认结果不是偶然现象，正式全量复现之前，我们先对 Mirai 做了三档规模的实验：100,000 行、250,000 行和 764,137 行（全量）。对应的 ROC-AUC 分别是 0.9512、0.9476 和 0.9482，变化幅度不大，说明指标已经基本收敛，也侧面验证了 KitNET 训练过程是稳定的。

在发现 6.2 节提到的 UCI 数据列错位问题之后，我们又把修复前后的 250,000 行实验各跑了一次，专门对比异常分数分布。修复前，p99.5 阈值下的 Recall 只有 0.61；修复后则升到 0.87，并和全量结果保持一致，这说明修复是有效的。

6.6.2 部署说明

为方便复现与扩展，本文将整个实验工作流脚本化，主要包括以下两类脚本。

单攻击实验脚本 `run_uci_feature_experiment.py` 通过命令行接收攻击名、数据路径、`grace` 长度和输出目录等参数，独立完成单个攻击的端到端复现，并输出 `summary.json`、`anomaly_scores.png`、`roc.png`、`pr.png` 和 `rmse.csv`。作为最小可执行单元，它既适合单独调试，也便于后续并行调度。

批量调度脚本 `scripts_run_overnight_full_reproduction.ps1` 用 PowerShell 编写，按 SSDP Flood → Mirai → OS Scan → SSL Renegotiation → ARP MitM → SYN DoS → Fuzzing → Active Wiretap → Video Injection 的顺序串行调用单攻击脚本，一次性把 9 类攻击都跑完。放在本地无人值守环境里，整个过程大约 6 小时就能得到全部结果。

日志归档：所有实验日志统一存放于 `logs/` 目录，按攻击名与时间戳分文件保存，便于后续追溯训练阶段的进度行、错误堆栈与耗时统计。

部署时我们尽量把约束写死，避免不同攻击之间的横向比较失真：9 类攻击统一使用同一组 grace 参数（FM=5000，AD=50000）和同一套评估口径（p99.5 阈值）；每次实验都放进独立子进程，尽量不让内存逐步累积；结果统一落成 CSV 和 PNG，方便后面聚合分析和写作。

第 7 章 总结

7.1 主要结论

本文以 Kitsune (NDSS 2018) 为对象，在 UCI 公开攻击数据集上做完了 9 类攻击的端到端复现。结果表明，Kitsune 在 Mirai、SSDP Flood、OS Scan、SSL Renegotiation 和 ARP MitM 这类显式异常攻击上总体有效，ROC-AUC 普遍高于 0.83，和原文趋势基本一致；但在严格的 p99.5 口径下，Mirai 等持续型攻击的 Recall (88.25%) 仍和论文里更乐观的数字有差距，而 Fuzzing、Active Wiretap、Video Injection 这类隐蔽攻击则基本失效。

在不重新训练 KitNET 的前提下，本文又加了基于 EWMA 的异常分数后处理，以 $O(1)$ /帧 的复杂度把持续型攻击的召回显著提高：Mirai 的 p99.5 Recall 从 88.25% 提升到 99.87%，SYN DoS 的 Recall 也提升了 100 倍以上，同时 FPR 仍保持在 0.005。换句话说，在 Kitsune 现有框架之上，加一层很轻的时序后处理，就已经能明显增强系统对长时序攻击的敏感度。

7.2 局限性

本文也有几个边界需要正视。首先，EWMA 对 SSDP Flood、ARP MitM 这类离散突发型攻击会出现召回回退，后面还得研究能否按攻击时间结构自适应地选 β 。其次，Kitsune 的特征工程 (AfterImage) 本身抓不到应用层语义，所以像 Video Injection 这类高语义攻击，不能指望只靠后处理去解决。再者，本文只覆盖了 UCI 数据集里的 9 类攻击，放到真实互联网流量里时分布鲁棒性还没有被完全验证。

7.3 未来工作

后续可以从四个方向继续推进：一是做基于攻击形态在线分类的混合阈值策略，持续型攻击启用 EWMA，突发型攻击保留原始分数；二是在 KitNET 上叠加更轻量的时序模型，比如 1D-CNN 或 GRU，从源头增强对长时序攻击的敏感度；三是在 CIC-IDS2017、TON_IoT 等更新数据集上重新复现并对比 Kitsune 与本文方案，看看跨数据集泛化能力如何；四是把系统接到真实在线流量采集环境里，再评估端到端延迟、吞吐和稳定性。

第 8 章 主要参考文献

- [1] Mirsky Y, Doitshman T, Elovici Y, et al. Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection[C]. In Proceedings of the Network and Distributed System Security Symposium (NDSS), 2018.
- [2] UCI Machine Learning Repository. Kitsune Network Attack Dataset[EB/OL]. <https://archive.ics.uci.edu/ml/datasets/Kitsune+Network+Attack+Dataset>, 2019.
- [3] Roesch M. Snort: Lightweight Intrusion Detection for Networks[C]. In Proceedings of the 13th USENIX Conference on System Administration (LISA), 1999: 229–238.
- [4] Breiman L. Random Forests[J]. Machine Learning, 2001, 45(1): 5–32.
- [5] Schölkopf B, Platt J C, Shawe-Taylor J, et al. Estimating the Support of a High-Dimensional Distribution[J]. Neural Computation, 2001, 13(7): 1443–1471.
- [6] Liu F T, Ting K M, Zhou Z H. Isolation Forest[C]. In Proceedings of the 8th IEEE International Conference on Data Mining (ICDM), 2008: 413–422.
- [7] Sakurada M, Yairi T. Anomaly Detection Using Autoencoders with Nonlinear Dimensionality Reduction[C]. In Proceedings of the 2nd Workshop on Machine Learning for Sensory Data Analysis (MLSDA), 2014: 4–11.
- [8] Meidan Y, Bohadana M, Mathov Y, et al. N-BaIoT: Network-Based Detection of IoT Botnet Attacks Using Deep Autoencoders[J]. IEEE Pervasive Computing, 2018, 17(3): 12–22.
- [9] Goodfellow I, Bengio Y, Courville A. Deep Learning[M]. Cambridge: MIT Press, 2016.

- [10] Sommer R, Paxson V. Outside the Closed World: On Using Machine Learning for Network Intrusion Detection[C]. In Proceedings of the 31st IEEE Symposium on Security and Privacy (S&P), 2010: 305-316.